耐量子計算機暗号

光成滋生

last update: 2026/11/10

概要

量子計算機による暗号解読

- 素因数分解, 離散対数問題など重要な公開鍵暗号の安全性に関わる問題が破られる危険性
  - 具体的なアルゴリズムについては後の講義で

耐量子計算機暗号

- 耐量子暗号, PQC (Post-Quantum Cryptography) とも
- 量子計算機が登場しても安全な古典計算機で動作する暗号技術

PQCへの移行

- 解読できなくても現在利用しているTLSを盗聴してデータを保存
- 将来量子計算機が登場したときに解読される危険性
- PQCへの移行は実用的な量子計算機の登場よりも前に進めるべき

NISTのPQCプロセス

標準化スケジュール

- 2016: NISTがPQCの標準化プロセスを開始
- 2017: 69方式受理
- 2019: 1st round : 26方式通過
- 2020: 2nd round : 15方式通過
- 2022: 3rd round : 4方式通過（KEMと署名が標準方式に確定）
  - 保留だった4方式が4th round評価へ
  - 2022 SIKEが攻撃された
  - 署名は格子暗号以外で追加公募, 確定したものはドラフト作成中
- 2024: 標準化文書公開

PQC標準化の状況

確定したもの

KEM

- Kyber → ML-KEM（格子）
- HQC (符号: 2025/3確定 2027年標準化予定)

署名

- Dilithium → ML-DSA（格子）
- SPHINCS+ → SLH-DSA（ハッシュ）
- Falcon → FN-DSA（FIPS206ドラフト, 格子）

その他の暗号の種類

符号暗号

- （標準には選ばれず）BIKE, Classic McEliece

同種写像

- SIKE, SIDH: 破れた(2022)
- SQIsign: 追加署名公募の2nd round中

多変数多項式

- 公開鍵が大きく署名が短い
- UOV/MAYO/QR-UOV: 同上

主なアルゴリズムの鍵サイズ

値のバイト数

アルゴリズム	種類	公開鍵	秘密鍵	暗号文・署名
ECDH	KEM	32	32	32
ECDSA	署名	32	32	64
ML-KEM768	KEM	1184	2400	1088
ML-DSA-65	署名	1952	4032	3309
SLH-DSA-128s	署名	32	64	7856

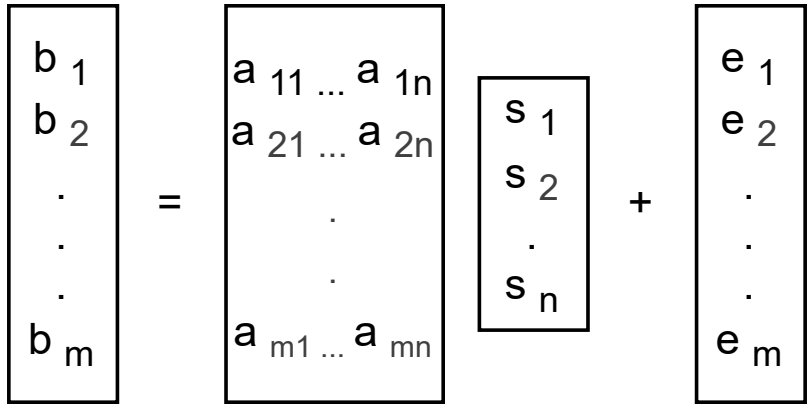
ブラウザのPQC対応（Cloudflare Research で確認できる）

- 2024/4: ChromeがX25519Kyber768（ドラフト）対応
  - 2024/11にML-KEM768に切り替わり, Edge/Firefoxも対応
- 2025/9: iPhoneのブラウザも対応
- 2026: ChromeのWeb Crypto APIがML-KEM/ML-DSAに対応（Chrome 155で正式予定）

格子暗号

LWE(Learning With Errors) 問題

- $M_{m,n}(\mathbb{F}_q)$ を縦 m, 横 n の $m \times n$ 次行列全体の集合とする
- $A \in M_{m,n}(\mathbb{F}_q)$ と縦（列）ベクトル $s \in {\mathbb{F}_q}^n$ を選ぶ
  $b := As + e \pmod q$ とし, (A, b): givenのとき s を求めよ
  - ここで e は小さい整数（後述）からなる m 次元縦ベクトル
  - $e = 0$ なら（$m \geq n$ として）普通の連立方程式を解くだけ
    - 注意: 表記の都合上, 行列やベクトルの縦横が他の書籍と転置しているかもしれない



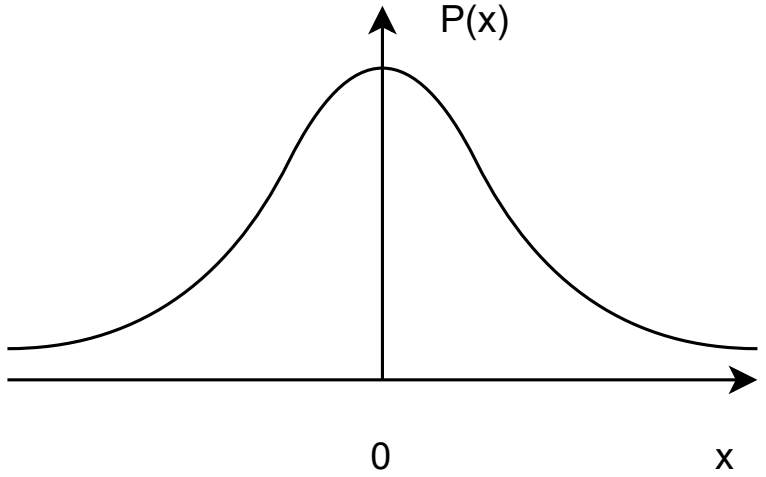
仮定

- 適切なパラメータに対してLWE問題は困難であることをLWE仮定という
- （判定版）LWE仮定
  - 上記の方法で生成した $\{(A, b)\}$ を（計算）LWE分布という
  - LWE分布と $(A, b) \underset{U}{\leftarrow} M_{m,n}(\mathbb{F}_q) \times {\mathbb{F}_q}^m$ を区別するのが困難

ノイズの分布

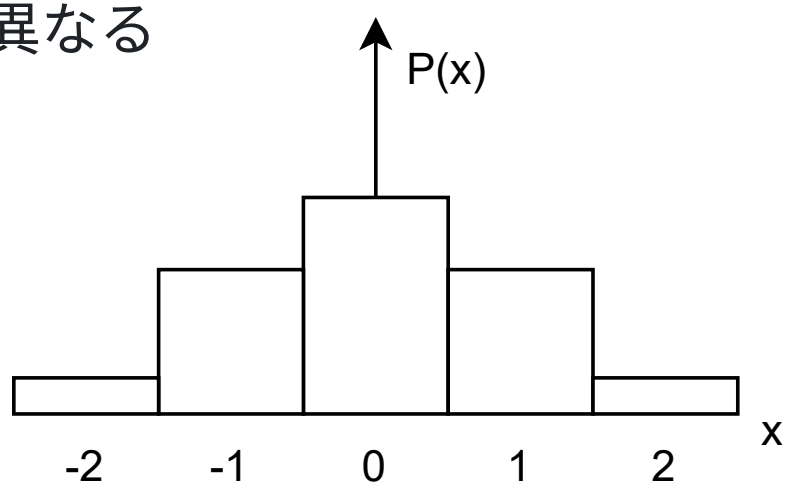
連続ガウス分布

- 平均0, 標準偏差 σ のガウス分布
  - $P(x) = \frac{1}{\sqrt{2\pi}\sigma}\exp(-x^2/2\sigma^2)$



離散ガウス分布

- 整数 x が $\exp(-x^2/2\sigma^2)$ に比例する確率で分布する
  - e は離散ガウス分布からサンプリングする
- 厳密には通常の連続ガウス分布の結果を整数に丸めたものとは異なる
- 扱いが困難なので実装時には適当な境界パラメータ L を決めて $[-L, L]$ の範囲で生成する（e.g., 二項分布）



Regev暗号

鍵生成

- $s \in {\mathbb{F}_q}^n, A \in M_{m,n}(\mathbb{F}_q), e \in {\mathbb{F}_q}^m$ を選び $b := As + e$ とする
  - s が秘密鍵で (A, b) が公開鍵

$m \in \{0, 1\}$ の暗号化

- $r \underset{U}{\leftarrow} \{0, 1\}^m$ を選び $c := (A^T r, b^T r + (q//2)m)$ が暗号文 $(q//2 := \texttt{floor}(q/2))$

$c = (u, v)$ の復号

- $t := [v - s^T u]_q$ が $|t| < q/4$ ならば $m = 0$, そうでなければ $m = 1$
  - ここで $[a]_q$ は $a \in [0, q-1]$ を $a \leq q//2$ なら a, その他は $a - q$ とする$(|[a]_q| \leqq q//2)$

正当性

- $(As + e)^T r + (q//2)m - s^T A^T r = e^T r + (q//2)m \approx (q//2)m$
  - $|e^T r| < q/4$ なら正しく復号できる

構造化格子

多項式環上の格子

- データサイズ削減のためにより効率のよい格子を利用する
- q: 素数, n: 2のべき, 1の原始 ν 乗根 $\zeta \in \mathbb{F}_q$ （位数が ν: $\zeta^\nu = 1$）
  - ML-KEM768では $q = 3329, n = 256, \nu = 256, \zeta = 17$
- $\phi(x) := x^n + 1, R := \mathbb{Z}[x]/(x^n + 1), R_q := \mathbb{F}_q[x]/(x^n + 1)$

MLWE (Module LWE) 問題

- $s \underset{U}{\leftarrow} {R_q}^k$ とノイズの分布 χ を選ぶ（ベクトルの内積 $s \cdot a = s^T a = a^T s$ に注意）

  MLWE分布: $A_{s,\chi} := \left\{ (a, s^T a + e) \in {R_q}^k \times R_q \mid a \underset{U}{\leftarrow} {R_q}^k, e \leftarrow \chi \right\}$

MLWE仮定

- MLWE分布と $(a, b) \underset{U}{\leftarrow} {R_q}^k \times R_q$ を区別するのが困難

Kyber PKEの概要

鍵生成

- $s \in {R_q}^k, A \in M_k(R_q), e \in {R_q}^k$ を選び $b := As + e$ とする
  - $sk := s$ が秘密鍵で $pk := (A, b)$ が公開鍵

暗号化

- Encode: 256bitの $m = \sum_i m_i 2^i$ から $\tilde{m} := \sum_{i=0}^{255} m_i (q//2) x^i \in R_q$ を作る
- 乱数 $r \in {R_q}^k, e_1 \in {R_q}^k, e_2 \in R_q$ を選び, $\tilde{r} := (r, e_1, e_2)$ として
  $Enc(pk, m; \tilde{r}) := (u, v) := (A^T r + e_1, r^T b + e_2 + \tilde{m})$ が暗号文

復号と正当性

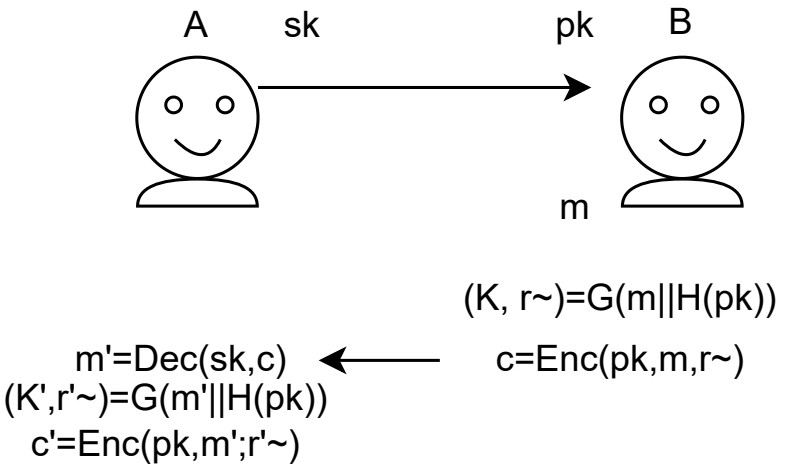
- $Dec(sk, (u, v)) := v - s^T u = r^T(As + e) + e_2 + \tilde{m} - s^T(A^T r + e_1)$
  $= (r^T As - s^T A^T r) + (r^T e + e_2 - s^T e_1) + \tilde{m}$
- Decode: $r^T e + e_2 - s^T e_1$ は小さいので $q//2$ に近いか否かで $\tilde{m}$ から m_i を0 or 1で復元
- 安全性: Kyber PKEはMLWE仮定の元でIND-CPA安全

ML-KEMの概要

Kyber PKEからIND-CCA2安全な鍵共有(Kyber-KEM)を実現

- 藤崎-岡本変換（の変種）をKEM単体に適用
  - 公開鍵を毎回使い捨てることでDH鍵共有と同じ通信回数
  - RSAは鍵生成（素数生成）が重たいので使い捨てにできない

鍵共有

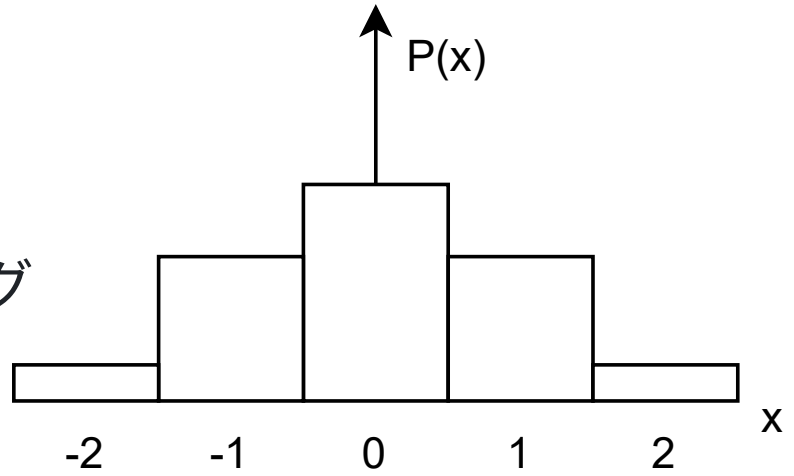


- A: (sk, pk) を鍵生成し pk をBに送り, 秘密の乱数 z を選ぶ
  - H, G, J: ハッシュ関数
- B: ランダムな m を選び $(K, \tilde{r}) := G(m \| H(pk))$ として
  $c := Enc(pk, m; \tilde{r})$ をAに送る. K が共有鍵
- A: $m' := Dec(sk, c), (K', \tilde{r}') := G(m' \| H(pk))$,
  $c' := Enc(pk, m'; \tilde{r}')$,
  $c = c'$ なら K' が共有鍵, そうでなければ $J(z \| c)$ を利用
- 注意: $c \neq c'$ のときエラーにしないのは復号の成否を漏らさないようにするため

ML-KEM768のパラメータと性能改善

主要パラメータ

- $n = 256, q = 3329 = 13n + 1, k = 3, \eta = 2$
  - $k = 2$（サイズ小, 安全性低）, $k = 4$（サイズ大, 安全性高）
- e は試行回数4の二項分布を2左に移動したものからサンプリング



公開鍵サイズの削減

- A は k^2 個の要素（256係数×12bit×9=3456B）
- SHAKE, SHA-3などからなるXOF, PRFを利用して256bitのseedから各種乱数を生成しAを決定

多項式乗算の高速化

- n 次多項式同士の乗算は素朴にやると $O(n^2)$ 回の演算が必要
- FFTの類似物である NTT (Number Theoretic Transform) を利用して高速化
- 多項式 $f(x) = \sum_i f_i x^i, g(x) = \sum_i g_i x^i$ の積 $h(x) = f(x)g(x) = \sum_i h_i x^i$
  $h_k = \sum_i f_i g_{k-i}$ を畳み込みという

FFT (Fast Fourier Transform) の概要

DFT (Discrete Fourier Transform)

- Fourier変換の離散版
- n を2のべき乗, $w := e^{2\pi\sqrt{-1}/n}$, 複素 n 次行列 $U := (u_{ij}) = (\sqrt{1/n} \cdot w^{ij})_{i,j}$ とする
- $(U\overline{U}^T)_{ik} = \sum_j u_{ij}\overline{u_{kj}} = (1/n)\sum_j w^{j(i-k)} = \delta_{ik}$ より U はユニタリ行列
  - δ_{ik} はクロネッカーのデルタ（$i = k$ なら1, それ以外は0）
  - $i = k$ のとき $\sum_j w^{j(i-k)} = \sum_j 1 = n$
  - $i \neq k$ のとき $\sum_j (w^{i-k})^j = (1 - w^{(i-k)n})/(1 - w^{i-k}) = 0$ （等比数列の和）
- 複素 n 次元ベクトル $a := (a_0, \ldots, a_{n-1})^T$ に対して
  $A := F_n(a) := Ua = (1/\sqrt{n})(\sum_i a_i w^{ij})_j$ をDFT,
  $F_n^{-1}(A) := \overline{U}^T A = (1/\sqrt{n})(\sum_i A_i w^{-ij})_j$ を逆DFTという

演算量

- n 次元ベクトルと行列の積なので定義に従うと n^2 回の乗算

計算の分解

添え字の偶数・奇数に分ける

- $m = n/2, n$ 次元の w を w_n とすると $(w_n)^2 = w_m, \sqrt{n/m} = \sqrt{2}$より
- $(\sqrt{n})A_j = \sum_{i=0}^{n-1} a_i {w_n}^{ij} = \sum_{i=0}^{m-1} a_{2i} {w_n}^{2ij} + \sum_{i=0}^{m-1} a_{2i+1} {w_n}^{(2i+1)j}$
  $= \sum_{i=0}^{m-1} a_{2i} {w_m}^{ij} + {w_n}^j \sum_{i=0}^{m-1} a_{2i+1} {w_m}^{ij}$
- $j = 0, 1, \ldots, m-1$ のとき $\sqrt{2}A_j = F_m(a_{\mathrm{even}})_j + {w_n}^j F_m(a_{\mathrm{odd}})_j$
- $j = m, m+1, \ldots, n-1$ のとき ${w_m}^j = {w_m}^{j-m}, {w_n}^j = -{w_n}^{j-m}$ より
  $\sqrt{2}A_j = F_m(a_{\mathrm{even}})_{j-m} - {w_n}^{j-m} F_m(a_{\mathrm{odd}})_{j-m}$
- $F_m(a_{\mathrm{even}}), F_m(a_{\mathrm{odd}})$ が求まれば $F_n(a)$ が求まる
  - 演算量は $F_m(a)$ 2回と ${w_n}^{j-m}$ の積 n 回で $2n^2/4 + n = n^2/2 + n$ 回

再帰的に分解

- もう一度分割すると $2((n/2)^2/2 + (n/2)) + n = n^2/4 + 2n$ 回
- これを繰り返すと約 $n \log_2 n$ 回で計算できる ⇒ FFTという

FFTを用いた多項式の乗算の高速化

多項式の係数の畳み込み

- n 次多項式 $f(x) := \sum_{i=0}^{n-1} a_i x^i, g(x) = \sum_{i=0}^{n-1} b_i x^i$ の積 $h(x) = f(x)g(x) = \sum c_i x^i$
- $c_k = \sum_{i=0}^{k} a_i b_{k-i}$ を畳み込みという: そのまま計算すれば $O(n^2)$ 回の乗算
  - 多項式を $x^n - 1$ で割った余りを考えると添え字は $\bmod n$ で計算できる
- $C_j := F(a)_j F(b)_j$ とすると
  $c_k := F^{-1}(C)_k = (1/\sqrt{n}) \sum_j F(a)_j F(b)_j w^{-jk} = (1/\sqrt{n^3}) \sum_{i,j,s} a_i w^{ij} b_s w^{sj} w^{-jk}$
  $= (1/\sqrt{n^3}) \sum_{i,s} a_i b_s (\sum_j w^{j(i+s-k)})$: $k = i + s$ となる組合せで先に加算する
  $= (1/\sqrt{n^3}) \sum_{i,s} a_i b_s n \delta_{k,i+s} = (1/\sqrt{n}) \sum_i a_i b_{k-i}$: 添え字は $\bmod n$ で計算
- つまりFFT変換して要素ごとに掛けて逆変換すれば畳み込みになる
  $\sum_i a_i b_{k-i} = \sqrt{n} F^{-1}(F(a)_j F(b)_j)_k$

演算量

- FFTが3回と n 回の乗算なので $O(n \log_2 n)$

NTTへの応用

有限体 $\mathbb{F}_q$ では $\exp(2\pi\sqrt{-1}/n)$ は使えない

- 代わりに $\mathbb{F}_q$ における1の原始 n 乗根 ω を利用するとFFTと同様に演算量の削減が可能

巡回畳込みと負巡回畳み込み

- 多項式の計算が $\mathbb{F}_q[x]/(x^n-1)$ の中だったら
  $c_k = \sum_{i=0}^{k} a_i b_{k-i} + \sum_{i=k+1}^{n-1} a_i b_{k-i+n} = \sum_i a_i b_{(k-i) \bmod n}$
  - 添え字を $\bmod n$ で計算する巡回畳み込みになる
- $\mathbb{F}_q[x]/(x^n+1)$ の中では $c_k = \sum_{i=0}^{k} a_i b_{k-i} - \sum_{i=k+1}^{n-1} a_i b_{k-i+n}$: 負巡回畳み込み

ML-KEM768でのNTT

- $\mathbb{F}_{3329}$ における1の原始512乗根が存在しない: $3329 - 1 = 13 \times 256$ は512で割り切れない
  - 1の原始 $n = 256$ 乗根 $\omega = 17$ を使う. $\omega^i \neq 1$ for $i \in [1, n), \omega^n = 1, \omega^{n/2} = -1$
- 不完全な負巡回畳み込みNTTを使い, 係数ごとの積の代わりに128組の2次式同士の積をする